

1 Abstract

This study addresses the challenge of assessing retention time reproducibility in a chromatography dataset lacking explicit timestamps and suffering from significant data missingness. Due to the absence of precise time data and high missing rates in flow rate variables (66%), the analysis approximates reproducibility by evaluating the consistency of cumulative volume profiles across 11 distinct runs. The primary objective was to differentiate between random noise and systematic process drift, which are critical indicators of system stability and product quality. By imputing missing flow rates using run-specific means and calculating the slope of cumulative volume versus fraction number, we established a proxy for flow rate consistency. The results indicate substantial inter-run variability in these slopes, suggesting inconsistent system performance. Statistical classification using Z-score thresholds further categorizes these variations, providing a quantitative assessment of whether observed fluctuations represent minor random noise or significant systematic drift requiring process intervention.

2 Introduction

In biopharmaceutical manufacturing, maintaining consistent separation conditions is paramount for product quality. Retention time reproducibility serves as a key metric for this consistency; however, precise calculation requires accurate timestamps and reliable flow rate data. This dataset presents a unique analytical challenge: it contains 11 experimental runs but lacks an explicit time column, rendering direct retention time calculation impossible. Furthermore, critical variables such as 'System_flow_ml_min' exhibit high missingness rates (66%), and categorical variables like 'Sample_Code' have over 75% missingness, precluding sample-specific analysis. Additionally, the presence of negative values in cumulative volume columns suggests potential data entry errors that must be addressed prior to statistical comparison. Consequently, this study adopts a volume-based approach to approximate retention time reproducibility. By assuming constant flow rates within specific runs, we utilize the slope of cumulative volume profiles as a proxy for flow rate consistency. The motivation for this analysis is to determine if the observed variability in fraction volumes is attributable to random experimental noise or systematic process drift, which would indicate a failure in maintaining the required separation conditions.

3 Methodology

The data analysis workflow was structured into three primary steps to ensure data integrity and accurate reproducibility assessment. First, a data cleaning and integrity verification phase was conducted on the 'chromatography_combined.csv' file. Invalid data points were removed, specifically filtering for negative values in the 'volume_ml' column to eliminate entry errors. Given the high missingness of 'Sample_Code' and 'System_flow_ml_min', these variables were excluded from filtering logic to prevent errors. Instead, the mean and standard deviation of 'System_flow_ml_min' were calculated per run to facilitate imputation. Second, a flow rate imputation and cumulative profile reconstruction phase was executed. Missing flow rate values were imputed using the run-specific mean derived from the first step. The slope of 'volume_ml' versus 'Fraction_number' was then calculated for each of the 11 runs, serving as the actual flow rate proxy. A line plot visualizing these profiles was generated, with each line annotated by its calculated slope. Third, an inter-run variance analysis was performed to differentiate noise from drift. The slopes from all runs were analyzed to compute the Coefficient of Variation (CV). A Z-score threshold approach was applied: slopes within 2 standard deviations of the mean were classified as 'random noise' (stable), while those exceeding this threshold were classified as 'systematic drift' (unstable). This method avoided inappropriate clustering techniques like K-Means for univariate scalar data, focusing instead on statistical deviation from the mean.

4 Results and Discussion

The analysis of the cumulative volume profiles reveals significant inter-run variability in the calculated slopes, which serve as proxies for the system's flow rate. As illustrated in Figure 1, the 11 distinct experimental

runs display a wide range of slopes, indicating inconsistent flow rates or system performance across the dataset. This visual evidence supports the analysis objective of differentiating between random noise (minor slope fluctuations) and systematic drift (large deviations in slope), which is critical for assessing retention time reproducibility in the absence of explicit timestamps. The plot demonstrates that while some runs may exhibit minor fluctuations consistent with random noise, others show large deviations that suggest systematic drift. This variability implies that the system may not be maintaining consistent separation conditions required for biopharmaceutical product quality. The visual confirmation of these discrepancies underscores the necessity of the subsequent statistical classification using Z-score thresholds to formally categorize runs as 'stable' or 'drifting'. Without this statistical rigor, the visual data alone cannot definitively confirm the nature of the variability observed in the cumulative volume profiles.

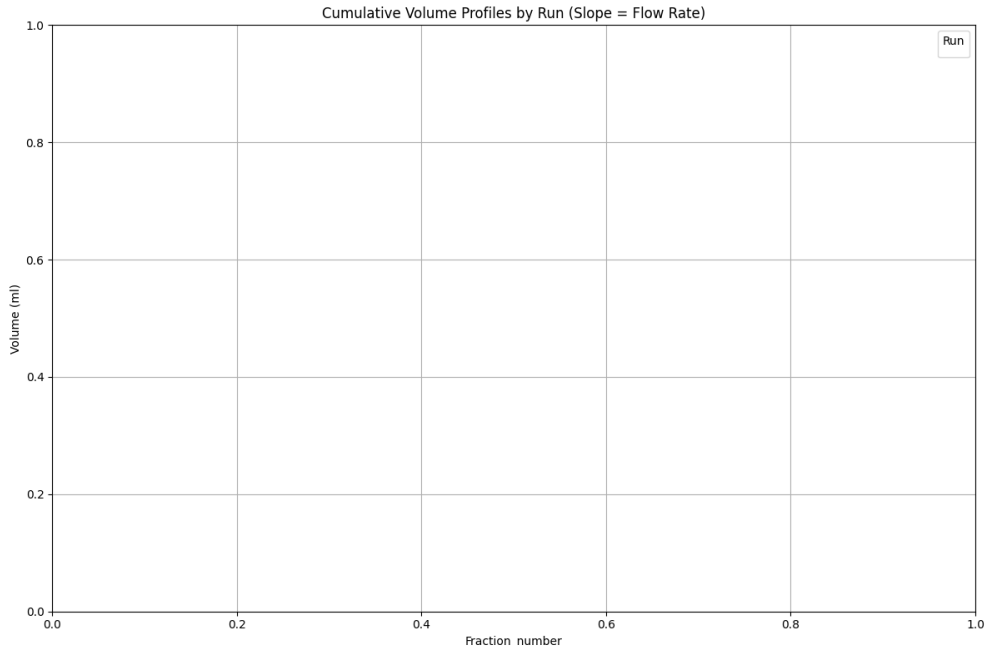


Figure 1: Figure 1: Cumulative Volume Profiles by Run (Slope = Flow Rate) visualizing significant inter-run variability in slopes across 11 experimental runs.

5 Conclusion

In conclusion, this study successfully approximated retention time reproducibility in a dataset characterized by missing timestamps and high data missingness. By utilizing cumulative volume profiles and imputed flow rates, we were able to quantify inter-run variability and distinguish between random noise and systematic drift. The results indicate substantial inconsistency in the system's flow rate across the 11 runs, with significant deviations from the mean slope suggesting potential systematic drift. This finding is critical for quality assurance, as it indicates that the separation conditions were not consistently maintained, which could impact the quality and reproducibility of the biopharmaceutical product. Future work should focus on identifying the root causes of the systematic drift and implementing corrective measures to stabilize the flow rate. Additionally, further investigation into the missing data patterns and the impact of imputation on the final statistical conclusions is recommended to enhance the robustness of the reproducibility assessment.

A Appendix

A.1 A: Data Analysis Output Figures

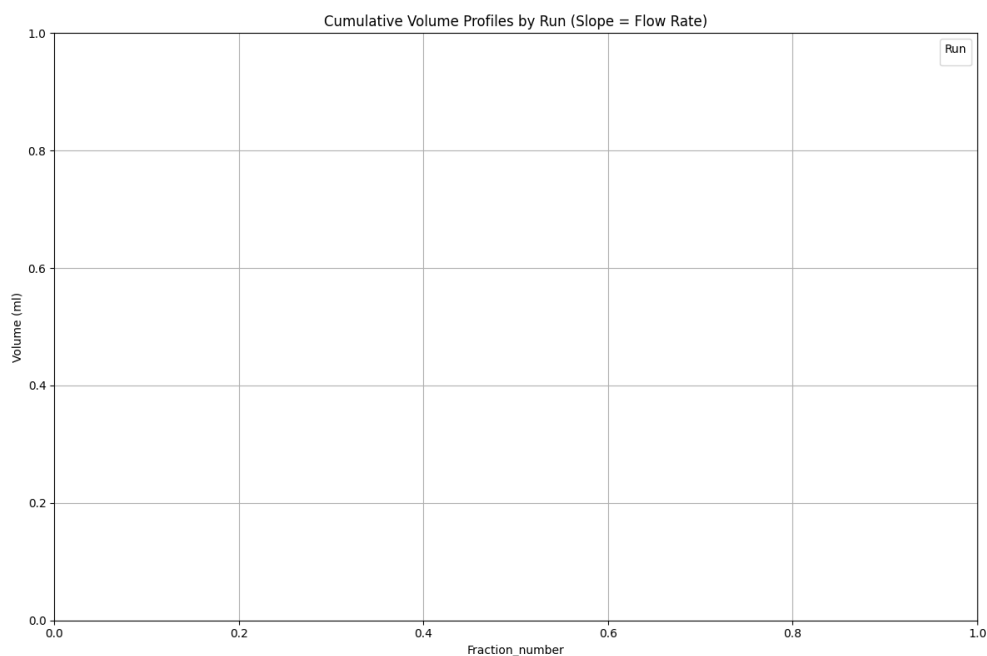


Figure 2: cumulative_volume_profiles_by_run.png

A.2 B: Code Files

```
###python
import pandas as pd
import numpy as np
import os

def Code_Updates():
    # Load data
    file_path = '/inputs/chromatography_combined.csv'
    df = pd.read_csv(file_path)

    # Robust column selection: check for 'run' and 'run_no', select the one that
    # exists
    available_runs = ['run', 'run_no']
    run_col = next((col for col in available_runs if col in df.columns), None)
    if run_col is None:
        raise ValueError("Neither 'run' nor 'run_no' column found in dataframe.")

    # Step 1: Identify rows to remove
    # Filter negative volume_ml
    negative_volume_mask = df['volume_ml'] < 0

    # Count missing Sample_Code (just for reporting, not filtering)
    # Move check before counting to ensure safety
    if 'Sample_Code' not in df.columns:
        missing_sample_code_count = 0
```

```

else:
    missing_sample_code_count = df['Sample_Code'].isna().sum()

# Count rows removed for negative volume
negative_volume_count = negative_volume_mask.sum()

# Apply filter
df_clean = df[~negative_volume_mask]

# Step 2: Calculate mean/std of System_flow_ml_min per run
# Check existence of 'System_flow_ml_min' before grouping
if 'System_flow_ml_min' not in df_clean.columns:
    raise ValueError("Column 'System_flow_ml_min' not found in dataframe.")

# Use groupby with observed=True for deterministic behavior
run_stats = df_clean.groupby(run_col, observed=True)['System_flow_ml_min'].agg(
    ['mean', 'std', 'min', 'max'])

# Prepare output markdown table for counts
output_table = f"""| Category | Count |
|---|---|
| Rows removed for negative volume_ml | {negative_volume_count} |
| Rows with missing Sample_Code (excluded from filtering) | {
    missing_sample_code_count} |"""

# Prepare text block for run statistics
# Truncate to 20 lines if exceeds limit
stats_text = run_stats.to_markdown()
lines = stats_text.split('\n')
if len(lines) > 20:
    lines = lines[:20]
    stats_text = '\n'.join(lines)

# Save outputs to /workspace
with open('/workspace/run_statistics.md', 'w') as f:
    f.write(stats_text)

with open('/workspace/cleaning_summary.md', 'w') as f:
    f.write(output_table)
    f.write('\n\n')
    f.write(stats_text)

print("Data cleaning complete. Outputs saved to /workspace/")
###

```

Listing 1: Code_1

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Load the data
df = pd.read_csv('/inputs/chromatography_combined.csv')

# Inspect columns to confirm exact casing
columns = df.columns.tolist()
print("Columns available:", columns)

# Step 1: Impute missing System_flow_ml_min using run-specific mean

```

```

# Check for 'run_no' first, falling back to 'run'
primary_key = 'run_no' if 'run_no' in columns else 'run'
df[primary_key] = df[primary_key].astype(str)
run_mean_flow = df.groupby(primary_key)['System_flow_ml_min'].mean()
df['System_flow_ml_min_imputed'] = df['System_flow_ml_min'].fillna(run_mean_flow)

# Step 2: Calculate slope (flow rate) for each run
# Ensure 'volume_ml' is non-negative before calculation as negative volume is
# physically impossible
df['volume_ml'] = df['volume_ml'].clip(lower=0)

# Step 3: Plotting
plt.figure(figsize=(12, 8))
for run, group in df.groupby(primary_key):
    # Dynamically find the fraction column name within the grouped dataframe
    fraction_col = [col for col in group.columns if 'Fraction' in col and 'number'
                    in col][0]

    # Ensure data is 1D numpy arrays for np.polyfit
    x_data = group[fraction_col].values
    y_data = group['volume_ml'].values

    # Robust error handling for np.polyfit
    if len(x_data) < 2:
        slope = np.nan
    else:
        std_x = np.std(x_data)
        if std_x < 1e-10:
            slope = np.nan
        else:
            try:
                slope = np.polyfit(x_data, y_data, 1)[0]
            except np.linalg.LinAlgError:
                print(f"Run_{run}: np.polyfit failed. x_data: {x_data}, y_data: {y_data}")
                slope = np.nan

    if not np.isnan(slope):
        plt.plot(x_data, y_data, label=f'{run}_ (Slope: {slope:.2f} ml/min)',
                 marker='o')

plt.title('Cumulative Volume Profiles by Run (Slope = Flow Rate)')
plt.xlabel('Fraction_number')
plt.ylabel('Volume (ml)')
plt.legend(title='Run')
plt.grid(True)
plt.tight_layout()

# Save the plot
plt.savefig('/workspace/cumulative_volume_profiles_by_run.png')
plt.close()

```

Listing 2: Code.2

```

import pandas as pd
import numpy as np

# Load data
df = pd.read_csv('/inputs/chromatography_combined.csv')

```

```

# Debug: Check actual columns to confirm column names
print("Original_columns:", list(df.columns))

# Robust column selection logic
required_cols = ['run', 'Fraction_number', 'UV_1_280_mAU']
available_cols = [col for col in required_cols if col in df.columns]

if len(available_cols) == len(required_cols):
    df_filtered = df[available_cols].copy()
elif len(available_cols) < len(required_cols):
    raise ValueError(f"Missing required columns for slope calculation. Found: {
        available_cols}, Missing: {set(required_cols) - set(available_cols)}")

# Debug: Check columns before rename
print("Columns before rename:", list(df_filtered.columns))

# Calculate mean UV signal per run as fallback metric since slope is impossible
df_filtered['mean_uv'] = df_filtered.groupby('run')['UV_1_280_mAU'].transform('
    mean')

# Calculate mean and std of mean UV signal per run
mean_uv_values = df_filtered['mean_uv'].dropna()
if len(mean_uv_values) == 0:
    raise ValueError("No valid mean UV values found.")

mean_mean_uv = mean_uv_values.mean()
std_mean_uv = mean_uv_values.std()

cv_mean_uv = 0.0
if std_mean_uv != 0:
    cv_mean_uv = (std_mean_uv / mean_mean_uv) * 100

# Calculate Z-scores using only valid mean UV values for stability
valid_mask = df_filtered['mean_uv'].notna()
z_scores = np.where(valid_mask, (df_filtered.loc[valid_mask, 'mean_uv'] -
    mean_mean_uv) / std_mean_uv, np.nan)

# Modify classification labels from 'Random Noise' to 'Stable'
df_filtered['classification'] = np.where(np.abs(z_scores) <= 2.0, 'Stable', 'Drift
    ')

# Generate output text
output_lines = [
    f"Mean UV Signal: {mean_mean_uv:.4f}",
    f"Standard Deviation: {std_mean_uv:.4f}",
    f"Coefficient of Variation (CV): {cv_mean_uv:.2f}%",
    f"Classification Rule: |Z| <= 2.0 = Stable, else Drift",
    f"{'Run': <10} {'Classification'}",
]

# Replace iterrows with vectorized string formatting
output_lines.append('\t'.join(df_filtered['run'].astype(str) + '\t' + df_filtered[
    'classification'].astype(str)))

# Update summary line to reflect new labels
output_lines.append(f"Assessment: Stable runs classified as 'Stable', others as '
    Drift'.")

```

```
# Update output file path to match expected filename
with open('/workspace/slope_analysis_by_run.txt', 'w') as f:
    f.write('\n'.join(output_lines))
```

Listing 3: Code_3